



UNIVERSITÉ DE
MONTPELLIER



STATISTIQUE
SCIENCE DES DONNÉES BIOSTATS
UNIVERSITÉ DE MONTPELLIER



FACULTÉ DES SCIENCES
DE MONTPELLIER

M2 STATISTIQUES ET SCIENCES DES DONNÉES – BIOSTATISTIQUES –

RAPPORT DE STAGE

Titre du stage

Guillaume BOULAND

Tuteur académique :

Nicolas Meyer

Tuteurs professionnels :

Mai Nguyen-Chi

Benjamin Charlier

Jury :

Ali Gannoun

Élodie Brunel

Nicolas Meyer



2023 – 2024

Remerciements

blabla

Résumé

Résumé

Abstract

Table des matières

1	Introduction	6
	Cadre du stage	6
	Contexte	6
	Question	6
	Objectifs	6
2	Segmentation automatique « framewise » pour le suivi de la réponse calcique dans les macrophages chez le zebrafish	8
2.1	Introduction	8
2.2	Éléments théoriques	9
2.2.1	<i>Image numérique</i>	9
2.2.2	<i>Différentes approches de segmentation d'images</i>	9
2.2.3	<i>Cartesian Genetic Programming (CGP)</i>	11
2.2.4	<i>Kartezio</i>	13
2.3	Évaluation des performances	14
2.4	Tracking de macrophages avec la librairie MacTrack2	15
2.4.1	<i>Création de modèles de segmentation</i>	16
2.4.2	<i>Visualisation de la qualité des modèles</i>	17
2.4.3	<i>Segmentation automatique de vidéos</i>	18
2.4.4	<i>Post-traitement</i>	19
2.4.5	<i>Analyse des signaux calciques</i>	19
2.4.6	<i>Limitations <- à enlever / modifier</i>	19
3	Suivi temporel des macrophages et fine-tuning d'un modèle de fondation	20
3.1	Introduction	20
3.2	Fine-tuning de SAM-2	21
3.2.1	<i>Définition</i>	21
3.2.2	<i>Jeu de données</i>	21
3.2.3	<i>Hyper-paramètres</i>	22
3.2.4	<i>Fonctions de perte</i>	23
3.2.5	<i>Optimisation</i>	25
3.2.6	<i>Résultats et comparaisons</i>	25
3.3	Propagation dans une vidéo	26
4	Conclusion	27

5	Matériel et Méthodes	28
----------	-----------------------------	-----------

A	Annexes	29
----------	----------------	-----------

A.1	Liste des fonctions de traitement d'images disponibles dans Kartezio	29
-----	--	----

Chapitre 1

Introduction

Cadre du stage

Contexte biologique

mehari . . .
lignée transgénique obtention des images, microscopie

Question

Objectifs

Les objectifs de ce stage sont multiples. Tout d'abord, nous cherchons à développer un outil de segmentation et de tracking des macrophages à partir de vidéos de microscopie obtenues par l'équipe, en effectuant une segmentation image par image dans un premier temps. Cependant, les macrophages étant des cellules mobiles et qui ont tendance à se rapprocher les uns des autres, à fusionner ou à se séparer, nous devons prendre ces paramètres en compte premièrement dans la segmentation mais aussi dans le tracking, c'est-à-dire dans le suivi temporel de la segmentation image par image. Puis, nous voulons, à partir de cette segmentation, étudier les flux calciques dans les macrophages au cours du temps. Avec cette analyse, nous cherchons à répondre à la question biologique posée, à savoir s'il existe une relation entre les flux calciques, et plus particulièrement les flashes calciques en mesurant les intensités de calcium dans chaque macrophage segmenté, et la polarisation des macrophages au cours du temps. De plus, nous souhaitons rendre cette étude accessible et compréhensible par les biologistes du laboratoire, à l'aide d'une documentation et de scripts intuitifs couplés à un jeu de données exemple ce qui les rend exécutables en un clic, afin qu'ils puissent continuer à analyser leurs vidéos de manière autonome à la suite de cette période. C'est pour ces raisons que nous utilisons des méthodes de segmentation automatique. L'outil développé, sous la forme de librairie Python, pourra aussi être utilisé en tant qu'aide à la segmentation de vidéos de microscopie en général, et pas seulement pour les macrophages, mais pour tout type d'imagerie biomédicale. La librairie développée, nommée MacTrack2, permet à la

fois d’analyser des vidéos de microscopie de macrophages à partir de modèles que nous fournissons, mais l’utilisateur a aussi la possibilité de créer les siens, étendant le domaine d’application de cette librairie.

La segmentation image par image est rapide, ce qui nous permet de traiter des vidéos entières en peu de temps. Cependant, les macrophages sont des cellules mobiles, il est donc nécessaire de bien suivre leur mouvement au cours du temps. C’est pour cela que nous explorons la possibilité de segmenter les macrophages directement à partir de la vidéo elle-même, en utilisant le modèle de fondation [1] SAM-2 [2], récemment développé par Meta AI en 2024. Cependant, ce modèle est généraliste et a été entraîné sur des données variées, nous cherchons donc à spécialiser ce modèle sur nos données, avec un fine-tuning des poids, pour qu’il puisse au mieux segmenter les macrophages dans les vidéos de microscopie.

Chapitre 2

Segmentation automatique « framewise » pour le suivi de la réponse calcique dans les macrophages chez le zebrafish

2.1 Introduction

Nous avons développé un outil qui permet de segmenter des macrophages dans des vidéos. Pour ce faire, nous avons tout d’abord réfléchi au problème de manière séquentielle, c’est-à-dire image par image (*framewise* en anglais). En poursuivant le travail déjà effectué au laboratoire par l’ancien stagiaire Axel de Montgolfier [3], étudiant en Master 2 Statistiques et Sciences des Données qui était présent durant l’année 2023-2024, nous avons amélioré le programme MacTrack créé par ses soins, en le rendant plus robuste avec notamment un meilleur jeu de données pour l’entraînement des modèles. Nous pouvons maintenant visualiser la qualité des modèles entraînés. Des scripts plus intuitifs pour les débutants, qui sont la cible de ce projet, ont été ajoutés ainsi qu’une documentation complète sur un site internet, disponible sur le dépôt GitHub du projet ([4]). Un post-processing a été implémenté pour corriger les erreurs de segmentation des macrophages. Ces erreurs que nous avons rencontrées étaient souvent dues à des problèmes au niveau de l’image de microscopie, comme par exemple la présence d’un fond bruité. Ce process sert aussi pour obtenir une segmentation continue dans le temps, sans changement de numérotation, afin d’obtenir la meilleure analyse possible des flux calciques présent dans ces macrophages.

Ainsi, toutes ces améliorations constituent **MacTrack2**, une librairie développée en Python, téléchargeable et utilisable simplement via les scripts et le jeu de données que nous fournissons en exemples. Elle se base principalement sur la librairie *Kartezio* [5], qui a été créée pour proposer une nouvelle approche de la segmentation d’objets dans des images microscopiques en se basant sur les principes du *Cartesian Genetic Programming* (CGP) [6]. À l’aide de cette méthode, *Kartezio* est capable de générer des pipelines de segmentations qui sont à la fois peu gourmands en ressources, notamment en quantité d’images pour l’entraînement

ou en temps computationnel, interprétables puisque cette librairie utilise une liste de fonction issues d’*OpenCV* [7, 8], une bibliothèque libre de fonctions spécialisées dans le traitement d’images, et performants.

2.2 Éléments théoriques

2.2.1 Image numérique

Les images que nous obtenons par microscopie confocale à l’aide de la lignée transgénique, comme vu dans l’**Introduction**, sont des images en 2 dimensions (2D) où l’on retrouve deux canaux de couleurs : le rouge pour la présence du macrophage (en bordure des cellules) et le vert pour la présence de calcium dans le cytoplasme du macrophage. Cependant, ces couleurs ont été ajoutées artificiellement par soucis de visualisation à l’aide du logiciel *ImageJ* [9] et plus particulièrement *Fiji* [10], qui ajoute un certain nombre d’options à ce logiciel, sous forme de plugins et macros. Ces couleurs ont été spécifiquement choisies car les protéines fluorescentes (*mCherry* pour le rouge et *GFP* (*Green Fluorescent Protein*) pour le vert) utilisées dans la lignée transgénique sont activées par la stimulation provoquée par un laser à une certaine longueur d’onde qui correspond à ces couleurs.

Les images que nous recevons, après avoir séparé les canaux, sont donc des images en niveaux de gris, où chaque pixel de celle-ci est un nombre réel entre 0 et 255, avec 0 représentant le noir et 255 le blanc. Cela nous évite la complexité de traiter une image RGB (Red, Green, Blue) où chaque pixel est un triplet d’intensité de chacune des trois couleurs (Fig. 2.1).

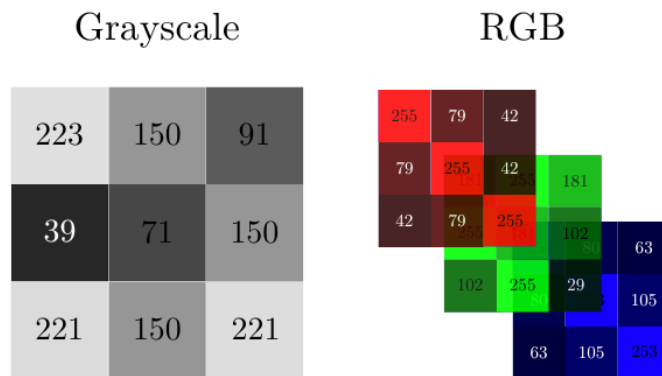


Figure 2.1. Représentations matricielles d’une image en niveaux de gris (*Grayscale*) et d’une image RGB. Figure issue de [11]

2.2.2 Différentes approches de segmentation d’images

La segmentation d’images est un domaine de la vision par ordinateur (*Computer Vision*) qui consiste à diviser une image en plusieurs régions ou objets d’intérêt. Ici, les objets d’intérêt sont des macrophages, mais la segmentation est un problème plus général, applicable à de nombreux domaines. Les régions ou objets d’intérêts

peuvent prendre plusieurs formes comme par exemple des rectangles permettant de délimiter la présence d'un objet. Elle sert à **détecter** tous les objets d'intérêt dans une image et peut même être utilisée pour labelliser ces objets [12, 13, 14, 15].

La **segmentation sémantique** est une autre approche de segmentation d'images où on affecte à chaque pixel une classe d'objet, et ce pour chaque objet de nature (ou sémantique) différente. Elle ne fait donc pas la différence entre deux voitures de couleur ou modèle différents par exemple, mais les classifera chacun comme étant une voiture.

La **segmentation d'instance** est une approche plus complexe qui se concentre sur la distinction des instances, c'est-à-dire des objets individuels, même les instances occluses de la même classe d'objet. Si on reprend notre exemple, les voitures sont maintenant segmentées individuellement, même si elles sont identiques, puisqu'elles sont vues comme des objets différents mais appartenant à la même classe « voiture ».

La **segmentation panoptique** combine les deux précédentes. Elle englobe à la fois la classification sémantique de chacun des pixels de l'image et la délimitation de chaque instance d'objet, mais son coût de calcul est plus élevé.

Dans notre cas, nous cherchons à segmenter des macrophages, donc sur le canal rouge de l'image, pour ensuite analyser les flux calciques de ceux-ci à partir de la segmentation obtenue. Nous avons donc besoin d'une segmentation d'instances car nous voulons détecter chaque macrophage tout en faisant attention à ne pas les confondre entre eux. De plus, nous avons une difficulté en plus car jusqu'à présent, nous avons seulement traité de segmentation d'images et non de vidéos. Or, les macrophages étant des cellules immunitaires très mobiles, ils pourraient se regrouper ou se séparer à tout moment de la vidéo. Donc la qualité de la segmentation est primordiale pour pouvoir au mieux retranscrire les comportements réels des macrophages.

figure avec trois images a des frames différentes pour des fusions separations...

Cependant, la segmentation d'instances fait appel à des modèles, les réseaux de neurones convolutifs par exemple, qui demandent des milliers d'images annotées, car il s'agit d'une méthode d'optimisation discrète. Il y a donc une perte empirique à minimiser pour qu'elle soit la plus faible possible. Or, annoter à la main un grand nombre d'images demande un temps énorme que nous n'avons pas. De plus, pour pouvoir segmenter autant d'images, il faut au préalable réaliser toutes les manipulations telles que la reproduction de poissons, le triage des œufs, la sélection des larves, la coupe de la nageoire caudale en fonction de la condition que l'on cherche à observer, l'observation et l'enregistrement microscopique pendant 40 minutes (durée des films que nous avons) à 3, 6, 12, 24, 48 et 72 heures post-amputation, pour chacun des poissons sélectionnés et pour finir le traitement des films sortis de microscopie pour qu'ils soient exécutables. Cela demande un coût humain et financier énorme que nous n'avons pas. En se tournant vers les modèles de classification sémantique, il avait été découvert l'année précédente *Kartezio* [5], une librairie Python, qui a pour promesse de segmenter, et ce en concurrençant les modèles basés sur des réseaux de neurones bien connus du domaine comme *Cellpose* [16, 17, 18], *Mask R-CNN* [19] ou encore *Stardist* [20], des objets dans des images biomédicales.

Plus précisément, *Kartezio* a pour avantage de générer des pipelines de segmentation robustes et interprétables à partir de peu d'images annotées, en se basant sur le CGP.

trouver un endroit pour parler de la différence de seg entre deux opératuer pour le type d'images qu'on a et mettre une figure qui compare les segmentations de moi et de norma, avec la quantification de la diffénrece entre les deux (75% par exemple)

2.2.3 Cartesian Genetic Programming (CGP)

La CGP [6], développée par Julian Miller initialement pour optimiser des circuits électroniques [21], est une approche de programmation génétique [22] (*Genetic Programming* (GP)) qui utilise des graphes pour coder des programmes informatiques (Fig. 2.2). La GP est un algorithme évolutif, c'est-à-dire qu'il reproduit les éléments de la biologie évolutive (génotype, génome, phénotype, sélection, recombinaison ou *crossover*, reproduction, mutation, ...) pour résoudre des problèmes d'optimisation discrète complexes. Le terme « cartésien » de la CGP vient du fait que les modèles générés sont représentés grâce à une grille de nœuds, où chaque nœud représente une fonction ou une opération. Chaque nœud est connecté à d'autres nœuds, formant ainsi un graphe orienté acyclique (Fig. 2.3). Le CGP est particulièrement adapté pour les problèmes de classification et de régression, mais il peut également être utilisé pour la segmentation d'images.

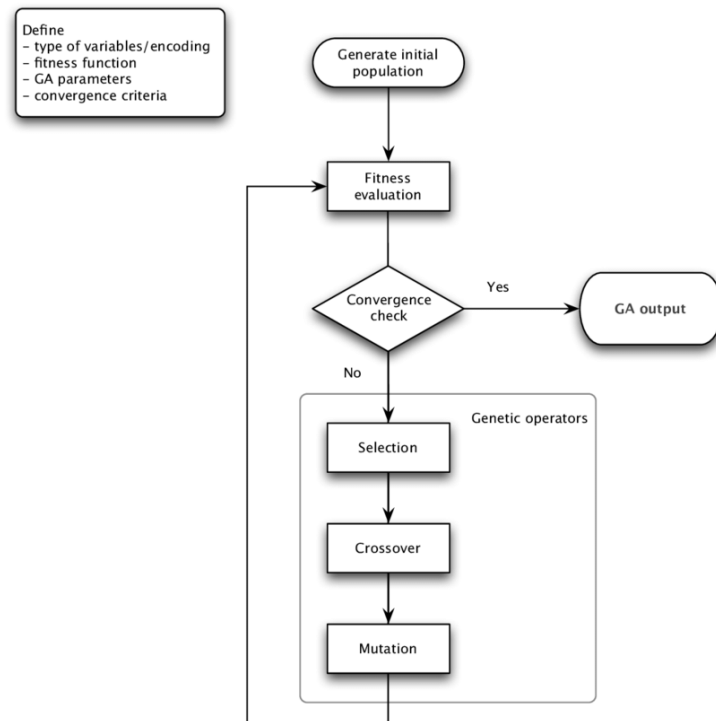


Figure 2.2

Chaque graphe peut être représenté par une chaîne d'entiers, qu'on appelle le génotype. Celui-ci est ensuite décodé pour obtenir le phénotype, qui est le programme

exécutable. Le phénotype est obtenu en parcourant le graphe, en commençant par les nœuds d'entrée (*input*) et en appliquant les fonctions de chaque nœud aux entrées correspondantes.

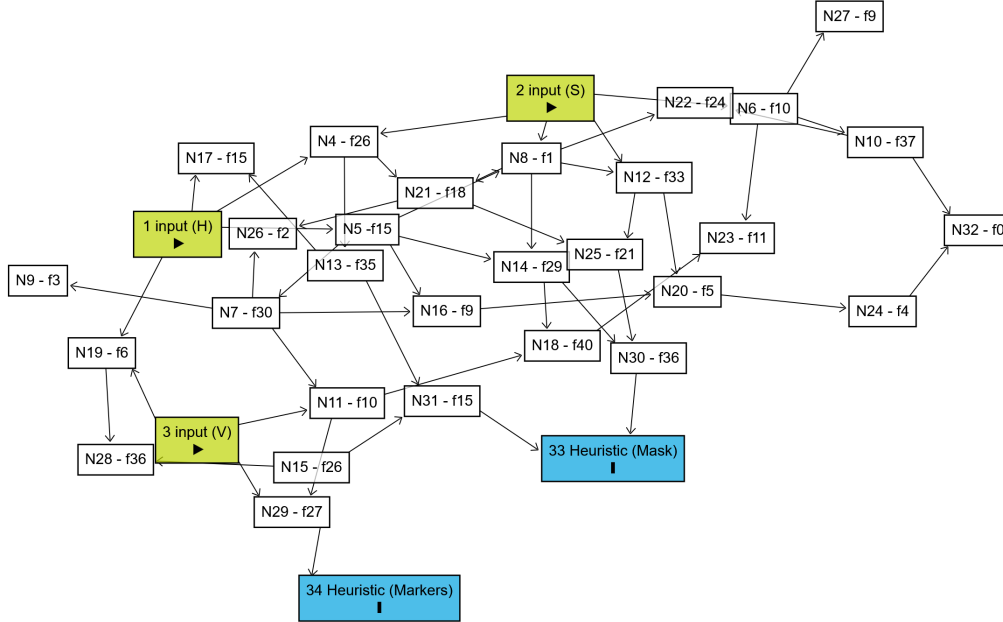


Figure 2.3. Exemple de graphe orienté acyclique de notre propre modèle (voir section **Tracking de macrophages avec la librairie MacTrack2**). Nx avec x le numéro du nœud et fy avec y le numéro de la fonction parmi une liste de fonctions (voir Tab. A.1 pour la numérotation des fonctions). Graphe obtenu avec Dagitty ([23]).

Un CGP utilise une stratégie évolutive (*evolutionary strategy* (ES)) qui va décider de quel opérateur génétique appliquer à chaque génération. Nous avons par exemple la stratégie $(1 + \lambda)$ où un individu (*parent*) génère λ descendants (*offspring*) par mutation et le meilleur de ceux-ci, selon une fonction d'adaptation (*fitness*), est conservé et deviendra l'individu parent de la génération suivante. Si les performances du meilleur descendant sont inférieures à celles du parent, alors la descendance complète est rejetée, on parle de remplacement strict. On peut aussi avoir une stratégie de mutation ciblée, où on altère aléatoirement les gènes (nœuds), en modifiant les fonctions et les connexions, avec un taux de mutation ciblée. Il existe une stratégie de sélection par tournoi, où l'on compare des sous-ensembles d'individus et on sélectionne le plus performant de chacun des groupes, qui est ensuite utilisé pour la reproduction. Il existe des stratégies hybrides qui combinent le CGP avec des méthodes d'apprentissage automatique comme le Deep Learning. Le CGP, contrairement à d'autres approches de GP, n'utilise pas vraiment de *crossover* et privilégie plutôt des mutations ciblées sur sa population d'individus. De plus, le CGP a une propriété de neutralité génétique, c'est-à-dire que certains changements dans le génotype ne seront pas forcément des changements visibles dans le phénotype. Cependant, ces nœuds servent à explorer l'espace de recherche d'une manière plus efficace. Certains nœuds, d'un autre côté, qui sont eux-mêmes efficaces peuvent être

facilement dupliqués et ré-utilisés. Cela nous permet d’obtenir des solutions plus compactes en restant tout autant efficaces. Dans le cadre de notre problème, qui est la segmentation d’images, cette particularité du CGP est très utile car souvent en segmentation d’images, la même opération est souvent utilisée plusieurs fois dans différentes parties de l’image.

La stratégie évolutive la plus utilisée est la stratégie $(1 + \lambda)$ car elle favorise la simplicité et la stabilité du processus d’évolution, tout en restant efficace et en limitant la taille de la population. La mutation reste l’opérateur génétique principal utilisé dans cette stratégie. C’est la même stratégie évolutive qu’utilise *Kartezio* pour générer des pipelines de segmentation d’images biomédicales.

2.2.4 Kartezio

Kartezio [5] est une stratégie computationnelle de segmentation d’images biomédicales se basant sur le CGP [6]. Disponible sous la forme d’une librairie Python, cette méthode est capable de générer des pipelines de segmentation d’images qui sont à la fois clairs et interprétables, en assemblant et paramétrisant des fonctions connues de traitement d’images issues de la librairie *OpenCV* [7, 8]. La liste de cette librairie disponible dans *Kartezio* est disponible en Annexe A.1 (Tab. A.1). Les fonctions de cette banque de fonctions de traitement d’images est la même qu’utilise le logiciel ImageJ [9], qui est un logiciel que les biologistes en général et plus particulièrement le laboratoire dans lequel j’étais utilisent pour traiter les images brutes de microscopie qu’ils obtiennent. Ainsi, l’explicabilité de *Kartezio* couplée à une banque de fonctions déjà connue des praticiens a poussé l’équipe, l’année précédente, à utiliser cette méthode pour segmenter les macrophages. Sachant que ce projet est destiné à être utilisé ultérieurement par l’équipe dans laquelle j’étais, il était nécessaire d’utiliser une méthode rapide, efficace, compréhensible et intelligible pour les biologistes.

Un autre avantage de cette méthode, qui utilise le CGP, est que le nombre de données (images) annotées nécessaires pour entraîner un modèle de segmentation est nettement réduit par rapport aux méthodes dites état de l’art que sont par exemple les modèles d’apprentissage profond (*Deep Learning*). *Kartezio* est donc une méthode que l’on appelle *Few-Shot Learning*, c’est-à-dire que l’on a besoin seulement de quelques images annotées pour entraîner un modèle qui ait des performances satisfaisantes. La Fig. 2.4 issue de l’article de Cortacero et al. [5] montre bien la structure de leur modèle, héritée du CGP.

demander combien il faut expliquer de la figure et si c’est pertinent d’expliquer, ou référer à l’article

Contrairement au CGP classique, *Kartezio* introduit le concept de nœud non évolutifs (*non evolvable*, voir Fig. 2.4) qui sont le *preprocessing*, qui permet de convertir l’image originale en un format préférentiel, ici le format HSV (*Hue, Saturation, Value* ou TSV en français pour Teinte, Saturation et Valeur) qui constitue ensuite respectivement les 3 premiers nœuds d’entrée : $\mathcal{N}_1$, $\mathcal{N}_2$ et $\mathcal{N}_3$. Cela permet d’uniformiser les images d’entrées. Ainsi, on a une méthode qui fonctionne pour tous les formats d’images classiques (png, jpg, ...) et même les plus flexibles comme le format TIFF (*Tag Image File Format*) par exemple, qui est un format

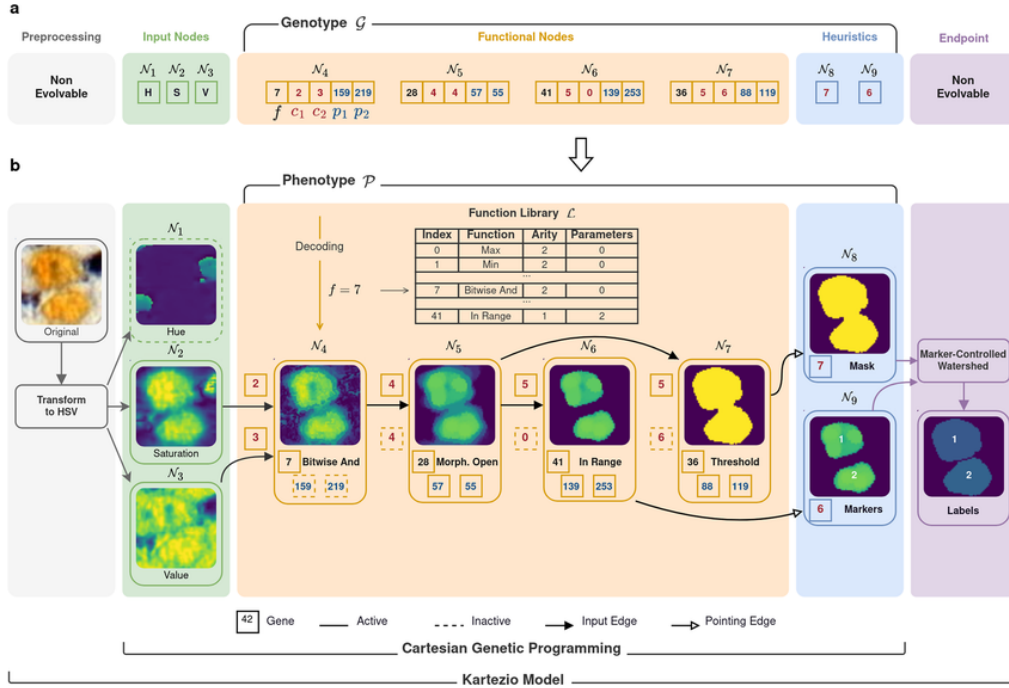


Figure 2.4

utilisé notamment en photographie pour le peu de perte de pixels qu'il engendre quand on modifie un tel fichier, mais aussi pour le fait qu'il supporte les images sur plusieurs couches comme les *z-stacks* qui sont beaucoup utilisés en microscopie in-vivo, comme c'est le cas pour nous. La dernière opération du pipeline est nommée *Endpoint* et est l'application d'une fonction choisie par l'utilisateur qui permet de créer un arrêt intentionnel des processus d'optimisation pour que celui des nœuds d'heuristique prenne place. Il fait partie intégrante du processus évolutif du génotype puisqu'il agit comme un point d'arrêt où à chaque génération on vérifie qu'il n'est pas dépassé (si c'est un seuil par exemple), mais il n'est pas voué à évoluer.

2.3 Évaluation des performances

À la fois pour Kartezio et donc pour le CGP et pour toute méthode qui vise à optimiser quelque chose, il est nécessaire d'avoir une métrique qui va nous permettre d'évaluer la qualité de l'apprentissage. Pour la segmentation d'images, la métrique la plus utilisée est l'**IoU** (*Intersection over Union*). On peut ainsi le définir comme étant le rapport entre l'intersection (en tant qu'aire) de la prédiction du modèle et de la vérité annotée et de leur union, pour un objet dans une image (Fig. 2.5) :

$$\text{IoU}_k(x, y) = \frac{|x \cap y|}{|x \cup y|}$$

avec x la prédiction de l'objet k et y la vérité annotée, et $|\cdot|$ la mesure de l'aire. Pour obtenir la valeur de l'IoU pour une image, on fait la moyenne sur tous les objets segmentés dans l'image :

$$\text{IoU} = \frac{1}{K} \sum_{k=1}^K \text{IoU}_k(x, y)$$

où K est le nombre total d'objet dans une image.

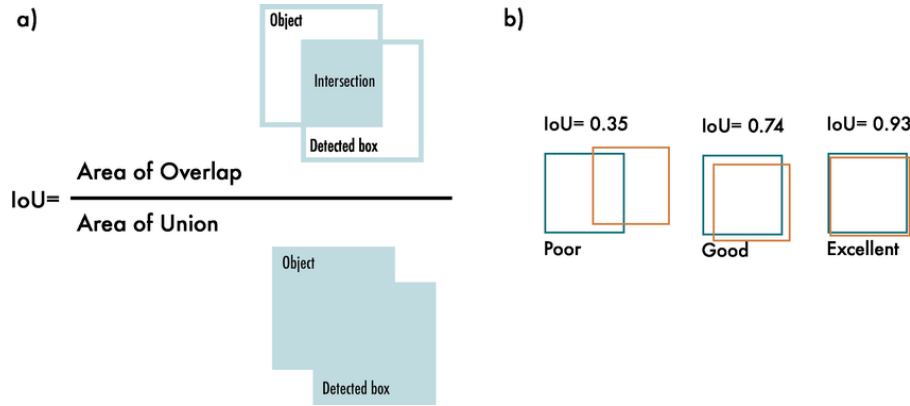


Figure 2.5. Schématisation de l'IoU (figure issue de [24]).

Il s'agit d'une valeur comprise entre 0 et 1, où 0 signifie que la prédiction ne recouvre pas du tout la vérité et où 1 signifie qu'elle la recouvre parfaitement. Plus on obtient une valeur d'IoU proche de 1 et plus on peut en déduire que la prédiction est bonne. Cependant, c'est une métrique qui a tendance à sous-estimer la qualité de la segmentation. En effet, un objet est mal segmenté mais qu'il recouvre une grande partie de la vérité, l'IoU aura une valeur convenable alors que la segmentation n'est pas si spécifique au problème.

Dans la pratique, obtenir une segmentation parfaite, signifiant que l'IoU vaudra 1, n'est tout simplement pas réaliste. Il est presque impossible que toutes les coordonnées (x, y) de la prédiction recouvrent toutes les coordonnées de la vérité. Ainsi, à travers l'IoU, on cherche donc à récompenser les prédictions qui se superposent bien avec la vérité. Ainsi il est considéré, dans la pratique, qu'un IoU supérieur à 0.5 est un IoU « acceptable » et on considère que si l'IoU dépasse 0.7, il s'agit d'un « bon » IoU (voir [25, 26]). Cette valeur est souvent utilisée comme seuil pour différencier les vrai positif des faux positifs ou faux négatifs. Par exemple, le jeu de données test *Pascal VOC* [27], qui est utilisé pour la détection d'objet et utilise l'IoU comme métrique principale, utilise un seuil de 0.5 ce qui signifie que toute détection d'objet ayant un IoU supérieur à 0.5 est considéré comme une détection positive (vrai positif). Il s'agit d'un consensus commun dans la communauté de la CV, construit sur un compromis entre qualité visuelle et quantitative.

2.4 Tracking de macrophages avec la librairie MacTrack2

présentation package basé sur stage année précédente

La librairie MacTrack2 est une librairie Python, deuxième version de MacTrack, développé par le stagiaire de l'année précédente.

2.4.1 Création de modèles de segmentation

La particularité de MacTrack2, et de Kartezio, est la facilité à entraîner des modèles. En effet, à l'aide d'un script simple, nous pouvons entraîner un modèle de segmentation d'images à partir d'un jeu de données d'images annotées. Nous avons, pour annoter nos données, utiliser ImageJ [9]. À l'aide de ce logiciel, nous avons pu détourer à la main chacun des macrophages dans les 25 images d'entraînement et 6 images de test que nous avons choisies à travers des vidéos à différents temps post-amputation (Fig. 2.6).

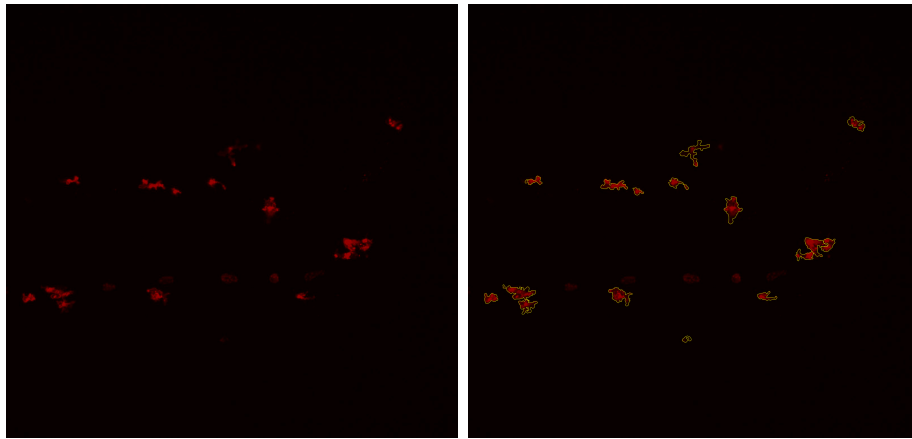


Figure 2.6

Les masques sont données sous la forme de fichiers ROI (Region Of Interest), qui sont présentés dans la Fig. 2.6, compilés sous un fichier ZIP. Ainsi il y a un fichier ZIP par image et un fichier ROI par macrophage détouré.

Pour cette trentaine d'image et pour obtenir un détourage précis, cela nous a pris environ 3 jours de travail. Les images que nous avons choisies, venues de vidéos différentes, sont des images pouvant être considérées comme représentatives de la diversité que l'on peut rencontrer lorsque l'on observe des macrophages. En effet, on a souvent des macrophages qui sont proches les uns des autres, on parle de chimiotactisme, mais aussi des macrophages qui ont des formes variées en fonction de leur état de polarisation. Par exemple, un macrophage dit M1 (pro-inflammatoire) aura tendance à être plutôt rond et compact, on peut l'observer à 3hpa (heures post-amputation), tandis qu'un macrophage M2 (anti-inflammatoire) aura tendance à être plus « filamenteux » et allongé, on peut l'observer à 24hpa (voir figure).

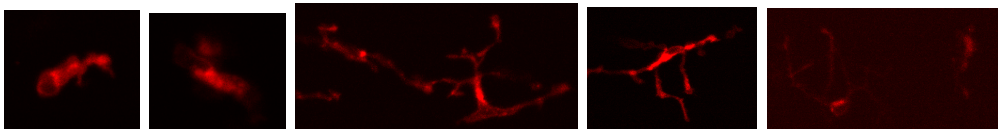


Figure 2.7. blabla (de gauche à droite : 3hpa, 6hpa, 24hpa, 48hpa, 72hpa).

Pour des questions de performances et de temps de calculs, nous avons choisi de créer 3 jeu de données identiques mais avec des tailles d'images différentes. Dans le premier, l'image était telle quelle, c'est-à-dire de la taille de l'image originale. Dans

le second, nous avons divisé la taille de l'image par 4, réduisant ainsi le nombre de pixels à traiter par image. Pour le troisième, nous avons retiré, dans les images, les zones noires qui ne sont pas utiles pour la segmentation. Dans le dernier cas, nous avons combiné les méthodes du deuxième et du troisième jeu de données. C'est ce dernier que nous avons gardé pour obtenir les modèles. C'est avec celui-ci que nous avons obtenu les meilleurs résultats dans des temps record. La Tab. 2.1 nous donne une idée des gains entre les différents jeux de données.

Modèle initial		Modèle petit		Modèle crop		Modèle petit+crop	
Train	Test	Train	Test	Train	Test	Train	Test
0.659	0.663	0.639	0.628	0.674	0.706	0.712	0.763
0.647	0.619	0.646	0.581	0.698	0.735	0.687	0.755
0.634	0.639	0.637	0.634	0.682	0.681	0.716	0.727
Temps de calcul							
Train	Test	Train	Test	Train	Test	Train	Test
1h27m	1s	36m30s	0s	49m48s	1s	9m0s	1s
1h28m	2s	5m39s	0s	1h	2s	4m6s	0s
1h54m	3s	7m35s	0s	1h44m	2s	3m48s	0s

Table 2.1. Exemples de performances des modèles entraînées sur les différents jeux de données. Les valeurs sont les IoU moyens sur le jeu de données d'entraînement et de test. Le modèle dont les performances sont écrites en rouge est celui que nous avons conservé pour la suite, et que nous avons fourni en exemple dans le dépôt GitHub du projet.

2.4.2 Visualisation de la qualité des modèles

Nous avons mis en place un outil permettant de visualiser la qualité des modèle entraînés. En superposant les prédictions données par le modèle et la vérité sur les images ayant permis d'obtenir ce dernier, nous pouvons visualiser la qualité de la segmentation qu'a le modèle que nous venons de créer. La Fig. 2.8 nous montre, pour une image venant de l'échantillon d'entraînement et une venant de l'échantillon de test, la superposition des prédictions du meilleur modèle, celui écrit en rouge dans la Tab. 2.1, et de la vérité annotée.

Les prédictions en rouge, obtenues en appliquant le modèle sur chacune des images des deux échantillons, sont parfois très petites en aire, on peut observer des petites taches considérées comme macrophages. Puisque le modèle est entraîné pour repérer des niveaux de gris sur une image, il n'y a pas de distinction réelle sur la question de savoir s'il s'agit bien d'un macrophage ou non.

C'est pour cela que la préparation des images au préalable est une étape primordiale à ne pas négliger. Préparées l'équipe sur le logiciel ImageJ, les images ont été soigneusement traitées pour faire en sorte de ne pas avoir d'artefacts dans le fond de l'image, et ainsi le fond a une moyenne de gris proche de 0.

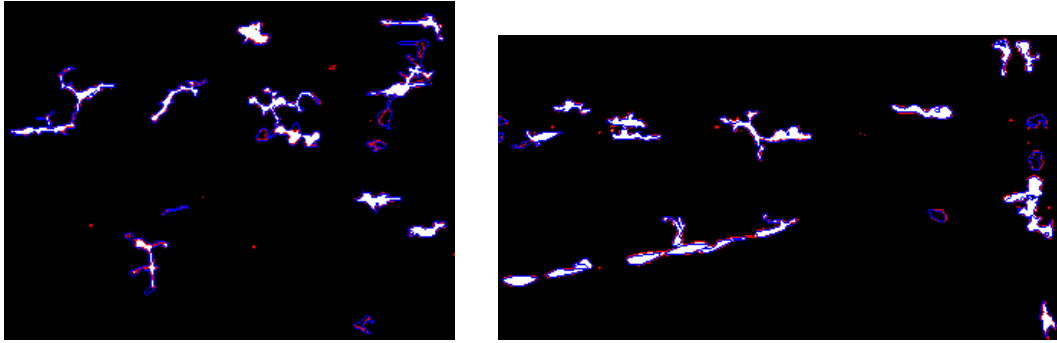


Figure 2.8. Exemples de comparaisons entre la vérité annotée (en bleu) et les prédictions du modèle choisi (en rouge) sur des images issues de l'échantillon d'entraînement (à gauche) et de test (à droite).

Le modèle choisi est celui écrit en rouge dans la Tab. 2.1. Pour l'image d'entraînement, l'IoU est de 0.67 et pour l'image de test, l'IoU est de 0.76. Ce sont toutes les deux des images issues du jeu de données « petit+crop ».

2.4.3 Segmentation automatique de vidéos

Une fois le modèle généré, visualisé et validé, il est possible de l'appliquer sur des vidéos entières. Lorsque les vidéos traitées par ma collègue sortent du logiciel, elles sont au format AVI et ont 266 images. Pour plus de détails sur les vidéos, voir **Matériel et Méthodes**. Nous commençons donc par convertir les vidéos des canaux rouge et vert et format mp4. Puis, nous découpons cette vidéo en images, les plaçons dans des dossiers spécifiquement rangés pour respecter la structure nécessaire au bon fonctionnement de Kartezio et créons automatiquement des fichiers pour garder cette structure. En effet, Kartezio s'attend à ce que les images d'entraînement soient présents dans un dossier « train » qui contient les images (dossier `train_x`) et les fichiers ZIP (dossier `train_y`) de chaque images contenant les masques (la vérité annotée) qui sont sous la forme de fichiers ROI. En revanche pour notre vidéo, les images extraites de celle-ci, pour le canal rouge qui nous est utile pour la segmentation des macrophages, sont placées dans le dossier « test », qui contient les images de la vidéo (dossier `test_x`), mais dans le dossier `test_y`, il n'y a pas de fichiers ZIP, car les vidéos que l'on cherche à segmenter automatiquement n'ont pas été annotées à la main. Cependant, pour la structure de Kartezio, il s'attend à ce qu'ils y ait des fichiers ZIP, au même nombre que les images dans le dossier correspondant. C'est pour cela que nous ajoutons automatiquement des fichiers ZIP vides dans le dossier `test_y`. Nous avons aussi besoin d'un fichier `dataset.csv` indiquant les images, qu'elles viennent de l'entraînement ou du test qui n'en est pas vraiment un, ainsi que les masques associés même s'ils sont vides, et finalement une colonne « set » qui indique si l'image est une image d'entraînement ou de test (donc issue de la vidéo).//

La segmentation de la vidéo peut donc enfin commencer après toute cette étape structurelle fastidieuse mais nécessaire. La première étape de segmentation est réalisée par la fonction `locate`. Elle permet d'obtenir la segmentation, à partir du modèle entraîné, de chaque image dans la vidéo. Il en sort à la fois les images segmenter mais aussi des sous-dossiers contenant les segmentations de chaque macrophage détecté image par image. Autrement dit, nous obtenons un dossier contenant 266

images en noir et blanc de la segmentation des macrophages, mais aussi un dossier contenant des sous-dossiers par macrophages, qui contiennent chacun les images de la segmentation de ce macrophage précis pour le nombre d'images dans lesquelles il a été repéré. La Fig. 2.9 nous montre un exemple des deux sorties obtenues à l'issue de cette première segmentation.

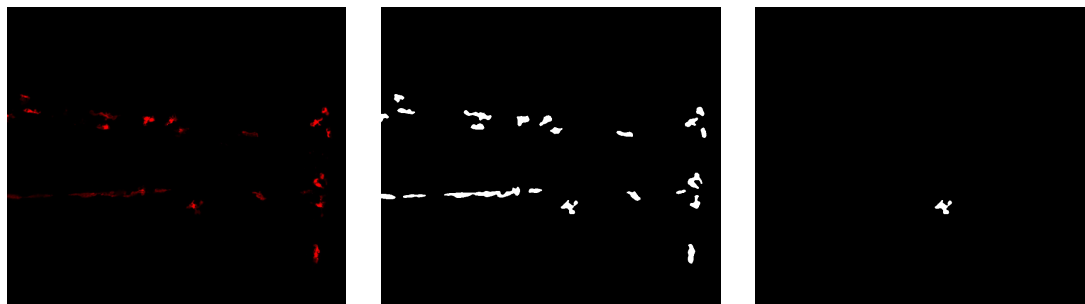


Figure 2.9. Illustrations des différentes sorties de la fonction `locate`, au milieu pour la segmentation complète et à droite pour la segmentation objet par objet, par rapport à l'image d'origine (à gauche).

Ces images proviennent de l'observation de la nageoire caudale d'un poisson à 3h post-amputation de celle-ci. Il s'agit de la deuxième image de la vidéo et des segmentations complète et d'un objet de cette même image.

différentes étapes
à partir d'un pipeline (le meilleur de ceux générés)

2.4.4 Post-traitement

- documentation et site internet, scripts automatiques pour création et exec de modèles avec dataset exemples
- post traitement pour corriger les erreurs de segmentation

2.4.5 Analyse des signaux calciques

le faire pour plusieurs vidéos

2.4.6 Limitations <- à enlever / modifier

seulement sémantique que l'on force à faire de l'instance, mais ça peut poser problème parfois et surtout on fait image par image, on a pas de mémoire réelle des images précédentes donc on cherche une méthode qui peut débloquent ce soucis

structure des entrees, fait que ct le travail d'axe et que j'ai mis 1 mois et demi a retrouver comment il avait fait marché ça, manque de docu ...

Chapitre 3

Suivi temporel des macrophages et fine-tuning d'un modèle de fondation

3.1 Introduction

SAM-2 (*Segment Anything Model 2*) est un modèle de segmentation d'images et de vidéos développé par Meta AI **sorti récemment, en 2024**. Il s'agit d'une version améliorée de la première version du modèle, SAM [28], généralisant le problème de segmentation d'images au domaine des vidéos (*VOS : Video Object Segmentation*). Entraîné sur plus de 50 000 vidéos pour 600 000 masques (annotations) **parler du type d'images et de vidéos, mettre le modele de fondation ici et passer a la ligne nv paragraphe**, SAM-2 est capable de segmenter des objets dans des vidéos de manière précise et efficace. L'architecture du modèle (voir Fig. 3.1) est une architecture basé sur les Transformers [29]. Elle comprend un *image encoder* qui est un Vision Transformer hiérarchique appelé Hiera [30], un *prompt encoder* qui prend en compte des clics (positifs ou négatifs), des boîtes ou des masques permettant de définir un objet d'intérêt dans une image et un *mask decoder* qui prend ces entrées et en ressort un masque de segmentation de l'objet sélectionné. Il utilise une mémoire en flux (*streaming memory*). Celle-ci permet de traiter les images d'une vidéo en temps réel, une par une. Ainsi, il peut garder en mémoire, à l'aide de la *memory attention* et de la *memory bank*, les objets et leurs interactions au cours du temps, permettant ainsi de générer des masques plus précis et de les corriger en fonction des images précédentes. La *memory attention* permet de « préparer » l'image en prenant en compte la *memory bank*, qui garde en mémoire les précédentes images, mais aussi les nouveaux points, masques, boîte, que l'utilisateur peut ajouter à chaque image.

Ainsi, SAM-2 est considéré (au même titre que SAM) comme un modèle de fondation (*foundation model*) [1], c'est-à-dire un modèle pré-entraîné sur une grande quantité de données, capable de généraliser à de nombreuses tâches différentes. Il est conçu pour être utilisé comme une base pour des tâches de segmentation en gardant de bonnes performances pour l'ensemble des applications de segmentation d'images et de vidéos.

Un des problèmes que nous rencontrons avec SAM-2 est que l'utilisateur doit, à la main, sélectionner des objets d'intérêt dans une ou plusieurs images et si durant

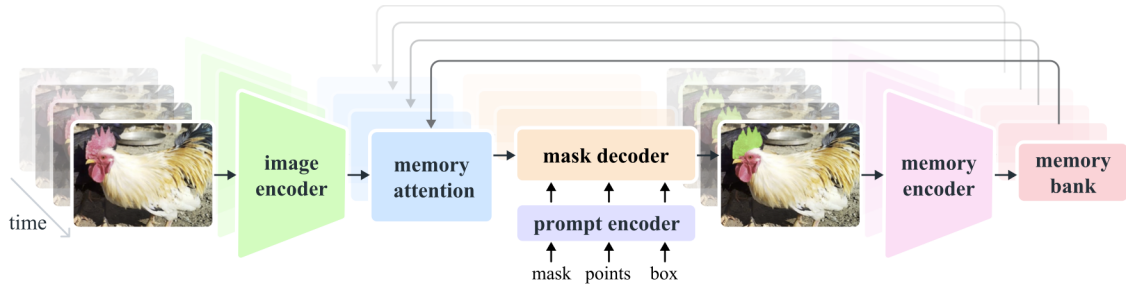


Figure 3.1. Architecture de SAM-2 (figure issue de [2]).

la vidéo un nouvel objet apparaît, il ne sera pas détecter tant que l'utilisateur ne l'a pas indiqué. Ainsi, nous allons utiliser la génération automatique de masques mise à disposition pour détecter les macrophages de manière automatique sur des images.

performances nulles donc fine tuning

3.2 Fine-tuning de SAM-2

3.2.1 Définition

Le *fine-tuning* (ou *affinage* en français) est une technique d'apprentissage supervisé qui consiste à modifier les poids d'un modèle pré-entraîné sur une tâche spécifique. Elle est souvent utilisé pour adapter un modèle généraliste à une tâche particulière. Ici, SAM-2 a été entraîné sur des photos qui n'ont rien à voir avec la problématique biologique que nous rencontrons. Par conséquent, afin d'obtenir une bonne segmentation des macrophages, il est nécessaire de modifier les poids du modèle pour obtenir une version spécialisée dans la détection de macrophages. Cela nous évite de devoir entraîner notre propre modèle en partant de rien, ce qui nous demanderait beaucoup trop de données que nous n'avons pas. Mais le nombre de données nécessaires pour obtenir un bon affinage n'est pas négligeable.

Le fine-tuning de SAM-2 agit seulement sur les poids du *prompt encoder* et du *mask encoder*, c'est-à-dire les parties du modèle qui prennent en entrée les points, masques, boîtes, et qui génèrent les masques de segmentation. Nous spécifions donc la prise en charge des entrées données par l'utilisateur, mais pas la partie qui encode l'image ni la partie qui mémorise les images précédente une fois le fine-tuning fini et que l'on cherche à segmenter une vidéo entière.

3.2.2 Jeu de données

Les annotations sont sous la forme d'images où chaque macrophage est détourné à la main sur le logiciel *ImageJ* (et plus particulièrement *Fiji* [10]). Les détournages sont ensuite séparés par label, afin de s'assurer qu'un macrophage ne soit pas confondu avec un autre (Fig. 3.2).

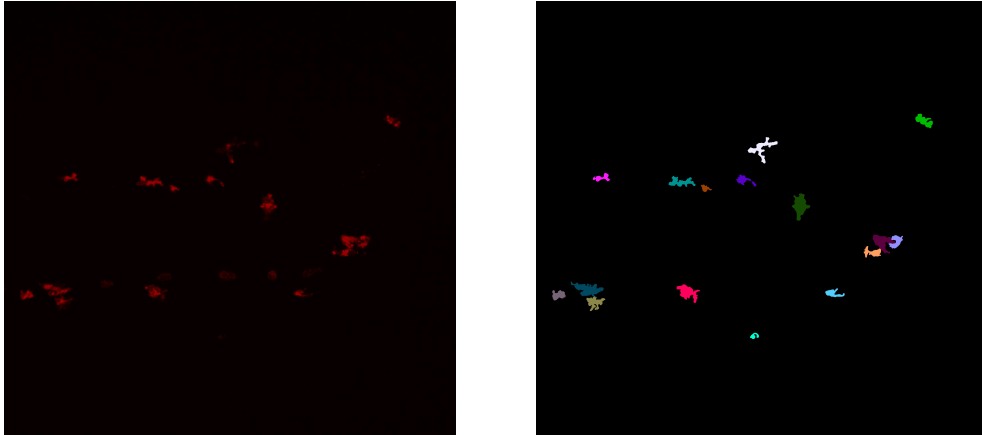


Figure 3.2. Image d'origine (à gauche) comprenant des macrophages et leur détourage (à droite).

Malgré que le fine-tuning de SAM-2 ne nécessite pas autant de données que l'entraînement d'un modèle de segmentation complet, il est quand même nécessaire de disposer d'un nombre conséquent de données. Ici, nous avons besoin d'images et de leur segmentation manuelle. Or, le temps imparti pour ce stage ne nous permet pas de composer un jeu de données complet qui satisferait ces besoins. Par conséquent, nous avons utilisé le programme MacTrack2 qui nous a permis de segmenter une vidéo de macrophages. Ainsi, nous disposons de 297 images et masques de macrophages (dont 266 faisant partie d'une seule vidéo) pour ré-entraîner les poids du modèle.

3.2.3 Hyper-paramètres

Les hyper-paramètres, dans un modèle, constituent les paramètres non appris par le modèle durant l'entraînement (ou le ré-entraînement). Ils sont fixés à l'avance et influencent son comportement et par extension ses performances. Les hyper-paramètres du pré-entraînement et de l'entraînement complet de SAM-2 sont disponibles dans leur papier [2] page 19.

Pour le fine-tuning, les sous-modules affectés sont le *mask decoder* et le *prompt encoder*. Nous utilisons l'optimiseur AdamW [31] avec un taux d'apprentissage (*learning rate*) de 0.0001 et une dégradation des poids (*weight decay*) de 0.00001, utile pour la régularisation L2. Le nombre total d'itérations est de 20 000, avec un nombre de pas pour l'accumulation des gradients (*accumulation step*) avant leur mise à jour égal à 4 steps. Le *scheduler* permet d'ajuster le *learning rate* durant l'entraînement pour améliorer la convergence du modèle. Ici nous utilisons un *stepLR scheduler*, ce qui signifie que tous les k steps, le *learning rate* est multiplié par un facteur γ . Nous avons $k = 500$ et $\gamma = 0.1$. Ainsi, le *learning rate* est réduit de 90% tous les 500 pas.

Contrairement à ce que l'on peut retrouver habituellement dans l'entraînement de modèles, la taille du batch est égale à une seule image. En effet, à chaque pas ...

3.2.4 Fonctions de perte

Pour entraîner un modèle, il est nécessaire de définir une fonction de perte $\mathcal{L}$ à minimiser. Elle est utilisée pour quantifier la qualité des prédictions du modèle par rapport à la « vérité » (ground truth). Dans le cas du fine-tuning de SAM-2, nous utilisons une combinaison de deux fonctions de perte : la *binary cross-entropy loss* qui sert de perte pour la segmentation et la *score loss* qui est utilisée en complément pour évaluer la qualité des prédictions des masques à partir de la métrique IoU (*Intersection over Union*). L'IoU est une métrique grandement utilisée dans les problèmes de segmentation d'images. Elle est définie, pour un macrophage k dans une image, de la manière suivante et peut être illustrée par la Fig. 2.5 :

$$\text{IoU}(M_k, G_k) = \frac{|M_k \cap G_k|}{|M_k \cup G_k|} = \frac{\sum_{i,j} M_k[i, j] \cdot G_k[i, j]}{\sum_{i,j} M_k[i, j] + \sum_{i,j} G_k[i, j] - \sum_{i,j} M_k[i, j] \cdot G_k[i, j]}$$

avec :

- $i \in \{1, \dots, H\}$ et $j \in \{1, \dots, W\}$ la hauteur et la largeur respectivement de l'image,
- $M_k = (M_k[i, j])_{i,j}$ le masque prédit pour l'objet k , une matrice binaire de taille $H \times W$ où chaque pixel vaut 1 s'il est dans le masque prédit, c'est-à-dire que si la probabilité que le pixel (i, j) appartienne au masque de l'objet k est supérieure à 0.5, sinon il vaut 0. On a donc :

$$M_k[i, j] = \begin{cases} 1 & \text{si } p > 0.5, \\ 0 & \text{sinon.} \end{cases}$$

avec p la probabilité que le pixel (i, j) appartienne au masque de l'objet k ,

- $G_k = (G_k[i, j])_{i,j}$ le masque de vérité pour l'objet k , une matrice binaire de taille $H \times W$ où chaque pixel (élément de la matrice) vaut 1 s'il est dans le masque de vérité, sinon il vaut 0,
- $|M_k \cap G_k| = \sum_{i,j} M_k[i, j] \cdot G_k[i, j]$ l'intersection entre le masque prédit et le masque de vérité, autrement dit le nombre de pixels en commun entre les deux masques,
- $|M_k \cup G_k| = \sum_{i,j} M_k[i, j] + \sum_{i,j} G_k[i, j] - \sum_{i,j} M_k[i, j] \cdot G_k[i, j]$ l'union entre le masque prédit et le masque de vérité, autrement dit le nombre de pixels dans au moins un des deux masques,
- $\sum_{i,j} M_k[i, j]$ le nombre de pixels du masque prédit,
- $\sum_{i,j} G_k[i, j]$ le nombre de pixels du masque de vérité.

En moyennant sur tous les objets k d'une image, on obtient la mIoU (IoU moyenne), métrique très utilisée, pour cette image :

$$\text{mIoU} = \frac{1}{K} \sum_{k=1}^K \text{IoU}(M_k, G_k)$$

Ainsi, nous pouvons définir la *score loss* comme suit :

$$\mathcal{L}_{\text{score}} = \frac{1}{K} \sum_{k=1}^K |s_k - \text{IoU}(M_k, G_k)|$$

avec K le nombre total de masques (donc de macrophages ou d'objets) dans l'image et s_k le score prédit pour l'objet k à partir de M_k .

Nous comparons donc le score prédit par le modèle pour chaque masque à la véritable qualité de chacun d'eux (l'IoU). Puisque'on cherche à minimiser cette perte, on aimerait que pour tout k , $s_k \simeq \text{IoU}(M_k, G_k)$ pour que la perte soit proche de 0.

Si $K = 0$, alors cela signifie qu'il n'y a pas de masques dans l'image, donc pas de macrophages. Un tel cas peut être rencontré, mais nous avons décidé de ne pas l'inclure dans le fine-tuning, car il n'apporte pas d'information utile concernant la segmentation des macrophages.

D'un autre côté, la *binary cross-entropy loss* $\mathcal{L}_{BCE}$ est une fonction de perte généralement utilisée dans des problèmes de classification binaire, et notamment dans les problèmes de segmentation d'images. Elle est utilisée pour quantifier la différence entre les prédictions du modèle et les véritables annotations (masques de vérité). Elle est définie comme suit :

$$\mathcal{L}_{BCE} = -\frac{1}{K \times H \times W} \sum_{k=1}^K \sum_{i=1}^H \sum_{j=1}^W y_{k,i,j} \log \hat{y}_{k,i,j} + (1 - y_{k,i,j}) \log(1 - \hat{y}_{k,i,j})$$

avec :

- $y_{k,i,j} \in \{0, 1\}$ la valeur du pixel (i, j) du masque de vérité pour l'objet k . On peut notamment remarquer que la matrice $G_k = (y_{k,i,j})_{i,j}$ est une matrice binaire (composée de 0 et de 1) de même dimension que les dimensions de l'image ($H \times W$).
- $\hat{y}_{k,i,j} \in [0, 1]$ est la probabilité que le pixel (i, j) appartienne au masque de l'objet k . Elle est obtenue par la transformation sigmoïde appliquée à la prédiction des masques du modèle, qui sont des logits générés par le *mask decoder* (voir Tab. 3.1). On a donc :

$$\hat{y}_{k,i,j} = \sigma(z_{k,i,j}) = \frac{1}{1 + \exp(-z_{k,i,j})}$$

avec $z_{k,i,j}$ le logit du pixel (i, j) du masque prédit pour l'objet k .

- H et W la hauteur et la largeur de l'image.
- K le nombre total de masques (donc de macrophages ou d'objets) dans l'image.

Interprétations fonction de perte ... ressemble a kullbackleibler mais différences ...

Logit ($z_{k,i,j}$)	Interprétation	Sigmoid ($\hat{y}_{k,i,j}$)
+10	Très sûr qu'il s'agit d'un objet	≈ 1
+1	Assez confiant qu'il s'agit d'un objet	≈ 0.75
0	Incertitude totale	0.5
-1	Assez confiant qu'il s'agit du fond de l'image	≈ 0.25
-10	Très sûr qu'il s'agit du fond de l'image	≈ 0

Table 3.1. Différents exemples des valeurs que peuvent prendre les sorties du *mask decoder* (logits), leur interprétation et la valeur de la probabilité associée (par transformation sigmoïde).

Finalement, la fonction de perte totale est une combinaison de ces deux dernières :

$$\mathcal{L} = \mathcal{L}_{BCE} + \lambda \mathcal{L}_{\text{score}}$$

avec λ un hyper-paramètre qui permet de pondérer l'importance de la *score loss* par rapport à la *binary cross-entropy loss*. Dans notre cas, nous avons fixé λ à 0.05.

3.2.5 Optimisation

Une fois la perte calculée, elle est divisée par le nombre de pas d'accumulation des gradients, soit 4 dans notre cas, comme défini dans la partie 3.2.3 introduisant les hyper-paramètres. Cela nous permet d'éviter de multiplier le gradient total par ce facteur. C'est utilisé lorsque l'on accumule des gradients sur plusieurs images (ici 4 fois une image) pour que la somme finale corresponde en réalité à une moyenne sur ce batch étendu. Ainsi, nous simulons un batch de taille 4 alors qu'en réalité nous n'avons qu'une seule image par step.

Ensuite, les gradients sont calculés en utilisant la méthode *backward* du package PyTorch [32], à partir de la perte totale $\mathcal{L}$ divisée par cet *accumulation step*. Puis, tous ces 4 steps, nous mettons à jour l'optimiseur AdamW. Le nombre de pas avant la mise à jour du *learning rate* est de 500. Ainsi, le *learning rate* est réduit de 90% tous les 500 pas. Ensuite, les gradients sont remis à zéro pour le prochain pas d'accumulation.

Pour finir, nous calculons l'IoU moyenne au pas ℓ comme étant :

$$\text{mIoU}_\ell = \begin{cases} 0.9 \cdot \text{mIoU}_{\ell-1} + 0.1 \cdot \frac{1}{K} \sum_{k=1}^K \text{IoU}(M_k, G_k) & \text{si } k > 0, \\ 0 & \text{sinon.} \end{cases}$$

3.2.6 Résultats et comparaisons

graphes miou et loss comp avant apres fine tune, performances et visuel

3.3 Propagation dans une vidéo

Chapitre 4

Conclusion

Disponibilité du code

Le code de MacTrack2 est disponible, ainsi que deux scripts détaillés pour la création de modèle et l'analyse d'une vidéo, avec un jeu de données d'exemple pour chacun des deux scripts, à l'adresse suivante : <https://github.com/guibouland/MacTrack2>.

Un site internet est disponible sur ce dépôt GitHub, regroupant toute la documentation et des instructions détaillées pour les utilisateurs plus novices, notamment les biologistes traitant la même problématique, qui sont la cible principale de ce projet. Il servira aussi aux futurs membres du laboratoire LPHI qui souhaitent utiliser MacTrack2 pour leurs propres projets.

Chapitre 5

Matériel et Méthodes

tests tests[5, 28, 2, 31]

Acquisition des images

microscope . . .norma merci

Format des vidéos

reconstitution sur imagej... par norma images toutes les 9sec donc sur 40 min de prise de vidéo on a que 266 images

Fine-tuning de SAM-2

L'optimiseur AdamW a été utilisé avec comme hyper-paramètres un taux d'apprentissage (*learning rate*) de 0.0001, une dégradation de pondération (*weight decay*) de 0.0001. L'entraînement a été effectué avec 20000 itérations. Nous avons utilisé un GPU Nvidia A10 Tensor Core avec 24 Go de mémoire.

Annexe A

Annexes

A.1 Liste des fonctions de traitement d'images disponibles dans Kartezio

Numéro	Fonction	Description
1	max	
2	min	
3	mean	
4	add	
5	subtract	
6	bitwise_not	
7	bitwise_or	
8	bitwise_and	
9	bitwise_and_mask	
10	bitwise_xor	
11	sqrt	
12	pow2	
13	exp	
14	log	
15	median_blur	
16	gaussian_blur	
17	laplacian	
18	sobel	
19	robert_cross	
20	canny	
21	sharpen	
22	gabor	
23	abs_diff	
24	abs_diff2	
25	fluo_tophat	
26	rel_diff	
27	erode	

Numéro	Fonction	Description
28	dilate	
29	open	
30	close	
31	morph_gradient	
32	morph_tophat	
33	morph_blackhat	
34	fill_holes	
35	remove_small_objects	
36	remove_small_holes	
37	threshold	
38	threshold_at_1	
39	distance_transform	
40	distance_transform_and_thresh	
41	inrange_bin	
42	inrange	

Table A.1. Fonctions et descriptions

Bibliographie

- [1] Rishi BOMMASANI et al. *On the Opportunities and Risks of Foundation Models*. 2022. arXiv : [2108.07258](https://arxiv.org/abs/2108.07258) [cs.LG]. URL : <https://arxiv.org/abs/2108.07258>.
- [2] Nikhila RAVI et al. « SAM 2 : Segment Anything in Images and Videos ». In : *arXiv preprint arXiv :2408.00714* (2024). URL : <https://arxiv.org/abs/2408.00714>.
- [3] Axel de MONTGOLFIER. *Github - Stage LPHI 2024*. URL : <https://github.com/Axeldmont/Stage-LPHI-2024>.
- [4] Guillaume BOULAND. *Github - MacTrack2*. URL : <https://github.com/guibouland/MacTrack2>.
- [5] Kévin CORTACERO et al. « Evolutionary design of explainable algorithms for biomedical image segmentation ». In : *Nature Communications* 14.1 (2023), p. 7112.
- [6] Julian MILLER et Andrew TURNER. « Cartesian Genetic Programming ». In : *Proceedings of the Companion Publication of the 2015 Annual Conference on Genetic and Evolutionary Computation*. GECCO Companion '15. Madrid, Spain : Association for Computing Machinery, 2015, p. 179-198. ISBN : 9781450334884. DOI : [10.1145/2739482.2756571](https://doi.org/10.1145/2739482.2756571).
- [7] OpenCV TEAM. *OpenCV : Open source Computer Vision Library*. <https://opencv.org/>.
- [8] OpenCV TEAM. *OpenCV : Open source Computer Vision Library - Python*. <https://github.com/opencv/opencv-python>.
- [9] Caroline A SCHNEIDER, Wayne S RASBAND et Kevin W ELICEIRI. « NIH Image to ImageJ : 25 years of image analysis ». In : *Nature methods* 9.7 (2012), p. 671-675.
- [10] Johannes SCHINDELIN et al. « Fiji : an open-source platform for biological-image analysis ». In : *Nature Methods* 9.7 (2012), p. 676-682. DOI : [10.1038/nmeth.2019](https://doi.org/10.1038/nmeth.2019).
- [11] NAMITA. *Convolutional Neural Networks*. Medium blog post. Déc. 2023. URL : <https://medium.com/@namitabagri/convolutional-neural-networks-fb0abfdc1c7c>.
- [12] Qing JIANG et al. *T-Rex2 : Towards Generic Object Detection via Text-Visual Prompt Synergy*. 2024. arXiv : [2403.14610](https://arxiv.org/abs/2403.14610) [cs.CV]. URL : <https://arxiv.org/abs/2403.14610>.
- [13] Tianhe REN et al. *Grounded SAM : Assembling Open-World Models for Diverse Visual Tasks*. 2024. arXiv : [2401.14159](https://arxiv.org/abs/2401.14159) [cs.CV].
- [14] Shilong LIU et al. « Grounding dino : Marrying dino with grounded pre-training for open-set object detection ». In : *arXiv preprint arXiv :2303.05499* (2023).
- [15] Tianhe REN et al. *Grounding DINO 1.5 : Advance the "Edge" of Open-Set Object Detection*. 2024. arXiv : [2405.10300](https://arxiv.org/abs/2405.10300) [cs.CV].

- [16] Carsen STRINGER et al. « Cellpose : a generalist algorithm for cellular segmentation ». In : *Nature methods* 18.1 (2021), p. 100-106.
- [17] Marius PACHITARIU et Carsen STRINGER. « Cellpose 2.0 : how to train your own model ». In : *Nature methods* 19.12 (2022), p. 1634-1641.
- [18] Carsen STRINGER et Marius PACHITARIU. « Cellpose3 : one-click image restoration for improved cellular segmentation ». In : *Nature Methods* (2025), p. 1-8.
- [19] Kaiming HE et al. « Mask r-cnn ». In : *Proceedings of the IEEE international conference on computer vision*. 2017, p. 2961-2969.
- [20] Uwe SCHMIDT et al. « Cell detection with star-convex polygons ». In : *Medical image computing and computer assisted intervention—MICCAI 2018 : 21st international conference, Granada, Spain, September 16-20, 2018, proceedings, part II* 11. Springer. 2018, p. 265-273.
- [21] Julian F MILLER, Peter THOMSON, Terence FOGARTY et al. « Designing electronic circuits using evolutionary algorithms. arithmetic circuits : A case study ». In : *Genetic algorithms and evolution strategies in engineering and computer science* 10 (1997).
- [22] M-J WILLIS et al. « Genetic programming : An introduction and survey of applications ». In : *Second international conference on genetic algorithms in engineering systems : innovations and applications*. IET. 1997, p. 314-319.
- [23] Johannes TEXTOR. *Dagitty*. <https://github.com/jtextor/dagitty>. 2010.
- [24] Juan TERVEN et Diana-Margarita CORDOVA-ESPARZA. *A Comprehensive Review of YOLO : From YOLOv1 to YOLOv8 and Beyond*. Avr. 2023. DOI : [10.48550/arXiv.2304.00501](https://doi.org/10.48550/arXiv.2304.00501).
- [25] Gaundez BOESCH. *What is Intersection over Union (IoU) ?* <https://viso.ai/computer-vision/intersection-over-union-iou/>. 2024.
- [26] Zhaowei CAI et Nuno VASCONCELOS. *Cascade R-CNN : Delving into High Quality Object Detection*. 2017. arXiv : [1712.00726](https://arxiv.org/abs/1712.00726) [cs.CV]. URL : <https://arxiv.org/abs/1712.00726>.
- [27] Mark EVERINGHAM et al. « The Pascal Visual Object Classes (VOC) challenge ». In : *International Journal of Computer Vision* 88 (juin 2010), p. 303-338. DOI : [10.1007/s11263-009-0275-4](https://doi.org/10.1007/s11263-009-0275-4).
- [28] Alexander KIRILLOV et al. « Segment Anything ». In : (2023). arXiv : [2304.02643](https://arxiv.org/abs/2304.02643) [cs.CV]. URL : <https://arxiv.org/abs/2304.02643>.
- [29] Ashish VASWANI et al. *Attention Is All You Need*. 2023. arXiv : [1706.03762](https://arxiv.org/abs/1706.03762) [cs.CL]. URL : <https://arxiv.org/abs/1706.03762>.
- [30] Chaitanya RYALI et al. *Hiera : A Hierarchical Vision Transformer without the Bells-and-Whistles*. 2023. arXiv : [2306.00989](https://arxiv.org/abs/2306.00989) [cs.CV]. URL : <https://arxiv.org/abs/2306.00989>.
- [31] Ilya LOSHCHELOV et Frank HUTTER. « Decoupled Weight Decay Regularization ». In : (2019). arXiv : [1711.05101](https://arxiv.org/abs/1711.05101) [cs.LG]. URL : <https://arxiv.org/abs/1711.05101>.
- [32] Adam PASZKE et al. « PyTorch : An Imperative Style, High-Performance Deep Learning Library ». In : (déc. 2019). DOI : [10.48550/arXiv.1912.01703](https://doi.org/10.48550/arXiv.1912.01703).